

Plan de Arquitectura e Implementación

Odoo 18: TPV y Desarrollo de Módulos

Integración de Hardware y Desarrollo de Autocompletado Custom

Este documento detalla el procedimiento técnico para desplegar el ecosistema de Punto de Venta (TPV) e Inventario en Odoo 18. Se incluye una sección especial dedicada al desarrollo in-house de un módulo de autocompletado de códigos de barras, garantizando un despliegue 100% libre de suscripciones utilizando el hardware PC Componentes.

1 Fase 1: Preparación de Infraestructura y Módulos Core

El PC "*PcCom Work Intel Core i5-12400/16GB*" actuará como un potente servidor local para Odoo 18.

- **Motor Base:** Instalar Python 3.10+ y PostgreSQL 14. Activar extensiones `unaccent` en la base de datos para búsquedas tolerantes a tildes.
- **Dependencia de Impresión:** Instalar el paquete de sistema `wkhtmltopdf`. Es imperativo para que Odoo renderice los tickets de compra y permita enviarlos a la impresora Approx appPOS80AM.
- **Módulos Comunitarios:** Desde el panel de Aplicaciones, instalar: **Inventario, Punto de Venta (POS), Contactos y Código de Barras (Community)**.

2 Fase 2: Integración Física de Escáneres y TPV

- **Lectores Honeywell y PcCom:** Conectar los lectores al equipo en modo *Keyboard Wedge* (Emulación de teclado).
- **Retorno de Carro (CR):** Configurar el Honeywell Voyager 1472g escaneando el código "Add CR Suffix" del manual. Esto inyecta un *Enter* automático, crucial para que Odoo procese el producto instantáneamente sin tocar el teclado.
- **Cajón Portamonedas:** Conectar el RJ11 del Approx CASH01 a la impresora térmica Approx. Odoo enviará la orden de apertura a través de la impresora al validar un pago en efectivo.

3 Fase 3: Desarrollo del Módulo Propio de Autocompletado (Alternativa API)

Para evitar pagar suscripciones externas (como Barcode Lookup), desarrollaremos un módulo personalizado (`custom_barcode_fetch`) que lea el código escaneado, consulte una base de datos gratuita de uso libre (como *OpenFoodFacts* para alimentación, o vuestra propia base histórica) y rellene los datos automáticamente.

3.1 1. Estructura del Módulo

En la carpeta addons de Odoo, crear la siguiente estructura de directorios:

```
custom_barcode_fetch/  
  __init__.py  
  __manifest__.py  
  models/  
    __init__.py  
    product_template.py  
  views/  
    product_views_inherit.xml
```

3.2 2. Lógica de Backend (Python)

El modelo `product.template` debe ser heredado. Usaremos el decorador `@api.onchange` para que, al disparar el escáner Honeywell en el campo de código de barras, Odoo dispare nuestra función en tiempo real.

```
1 from odoo import models, api  
2 import requests # Libreria para consumir APIs gratuitas  
3  
4 class ProductTemplate(models.Model):  
5     _inherit = 'product.template'  
6  
7     @api.onchange('barcode')  
8     def _onchange_barcode_fetch_data(self):  
9         if self.barcode and len(self.barcode) >= 8:  
10             # Ejemplo integrando OpenFoodFacts (API 100% Gratuita)  
11             url = f"https://world.openfoodfacts.org/api/v0/product/{self.barcode}.  
12             json"  
13             try:  
14                 response = requests.get(url, timeout=5)  
15                 data = response.json()  
16  
17                 if data.get('status') == 1:  
18                     product_info = data.get('product', {})  
19                     # Autocompletar el nombre del producto  
20                     if product_info.get('product_name'):  
21                         self.name = product_info['product_name']  
22                     # Odoo rellenara el formulario frente al usuario  
23             except Exception as e:  
24                 pass # Si falla, se rellena a mano sin bloquear el sistema
```

Listing 1: models/product_template.py

3.3 3. Modificación del Frontend mediante XPath (XML)

Para que el usuario tenga un botón manual en caso de que quiera forzar la búsqueda, heredamos la vista original utilizando XPath para inyectar nuestro código de forma segura y mantenible.

```
1 <?xml version="1.0" encoding="utf-8"?>  
2 <odoo>  
3     <record id="view_product_form_inherit_custom_barcode" model="ir.ui.view">  
4         <field name="name">product.template.form.inherit.custom</field>  
5         <field name="model">product.template</field>  
6         <!-- Referencia a la vista original de Odoo -->  
7         <field name="inherit_id" ref="product.product_template_only_form_view"/>
```

```

8      <field name="arch" type="xml">
9
10         <!-- Localizamos el campo barcode e insertamos un elemento DESPUES -->
11         <xpath expr="//field[@name='barcode']" position="after">
12             <span class="text-muted oe_inline"
13                 invisible="not barcode">
14                 (Búsqueda autocompletada activada)
15             </span>
16         </xpath>
17
18     </field>
19 </record>
20 </odoo>

```

Listing 2: views/product_views_inherit.xml

4 Fase 4: Reglas Nativas de Nomenclatura

Una vez que el módulo asigna el código base, Odoo debe saber interpretar variaciones.

- **Nomenclatura EAN-13:** Activar desde Ajustes > Inventario > Escáner de código de barras.
- **Productos al peso:** Si el escáner lee 2112345015002, el motor nativo de Odoo extraerá el prefijo 21, buscará el ID 12345 y aplicará una cantidad automática de 1.500 Kg directamente en el albarán o ticket del TPV, sin necesidad de que el empleado utilice el teclado.

5 Fase 5: Configuración Operativa del TPV

El *Point of Sale* en Odoo 18 opera con un DOM reactivo ultrarrápido (OWL Framework).

- **Arqueos de Caja:** Odoo 18 exige la declaración de fondo de caja al abrir la sesión. Este registro queda encriptado en el backend para la auditoría de ventas.
- **Creación en Caliente:** Si el módulo personalizado anterior no encuentra un producto, el cajero puede hacer clic en "Crear Producto" directamente desde la interfaz de cobro, disparar el escáner y asignar un precio en 5 segundos sin abandonar al cliente que está en caja.